Lecture 2 — Principles of Big Data Systems

CS4225 / CS5425 · Big Data Systems for Data Science · National University of Singapore

1 What Are the Principles of Big Data Systems?

Core insight: Big data systems are designed around hardware reality. They assume data is huge, machines fail, and communication is expensive, so good system design prioritizes scalability, locality, and fault tolerance.

This lecture focuses on practical principles behind systems like Hadoop and Spark: how data centers are organized, why latency and bandwidth matter, and why abstractions are needed for reliable large-scale parallel processing.

Context from Lecture 1

Big data problems have high volume, velocity, and variety, often with uncertain veracity. Traditional single-machine processing struggles at this scale.

Goal of Lecture 2

Build intuition for system-level decisions: scale out vs scale up, move compute to data, sequential access over random access, and framework-driven distributed execution.

2 Why These Principles Matter

At web scale, even simple operations become expensive. Scanning hundreds of terabytes or coordinating thousands of machines requires system-level design decisions, not just faster code.

Motivation: The challenge in big data is usually not one algorithmic trick. It is engineering the full execution path so that compute, storage, and network bottlenecks are all handled.

Cloud Model	What You Manage	Example
IaaS	VM, OS, runtime, application	Amazon EC2, Google Compute Engine
PaaS	Mainly application logic	Google App Engine
SaaS	Just use the software	Gmail, Dropbox

Utility computing: You rent compute as needed, enabling elasticity: scale up during spikes and scale down when demand falls.

3 Data Center as the Computer

Key idea: Instead of treating one server as the main computer, modern systems treat the whole data center as one distributed computer.

A typical hierarchy is server → rack switch → datacenter switch. Data transfer cost depends heavily on where data lives in this hierarchy.

```
Server A DRAM  → local, very fast
Server A Disk  → local but much slower
Server B DRAM  → same rack: add rack-switch cost
Server C DRAM  → different rack: add rack + datacenter-switch cost
```

Exam tip: Always reason from topology first: same machine, same rack, or different rack. This often determines the bottleneck.

4 Latency, Bandwidth, and Throughput

Term	Meaning	Intuition
Latency	Delay to start/complete transfer	How long before you see first response
Bandwidth	Maximum transfer rate	How wide the pipe is
Throughput	Actual achieved transfer rate	What you really get in practice

Path Rule 1

Latency is approximately additive across hops. Multiple links/switches increase end-to-end delay.

Path Rule 2

Bandwidth is constrained by the slowest segment (bottleneck). One weak link limits the whole path.

Practical takeaway: For small data, latency often dominates; for large data, bandwidth usually dominates.

5 Storage Hierarchy and Access Cost

Storage	Typical Latency	Typical Bandwidth	Capacity/Cost Pattern
DRAM	~100 ns	~20 GB/s	Small and expensive
SSD	~100 µs	~1 GB/s	Middle ground
Hard disk	~10 ms	~200 MB/s	Large and cheap

Important gap: Disk latency is about 100,000× slower than DRAM latency (10 ms vs 100 ns).

Disk Access Pattern	Effective Performance	Why
Random read	~400 KB/s for 4 KB reads	Seek latency paid repeatedly
Sequential read	~200 MB/s	Continuous transfer with few seeks

Design implication: Sequential scan can be hundreds of times faster than random access, so big data systems are designed to process data in large sequential chunks.

6 Data Flow Cases and Performance Intuition

Use these four scenarios to estimate cost quickly:

Case 1
Program → Local DRAM
Latency ~100 ns
Bandwidth ~20 GB/s

→

Case 2
Program → Local Disk
Latency ~10 ms
Bandwidth ~200 MB/s

→

Case 3
Program → Rack Switch → Remote DRAM
Latency ~600 µs
Bandwidth ~100 MB/s

→

Case 4
Cross-rack Remote DRAM
Latency ~1600 µs
Bandwidth ~10 MB/s

Scenario	Main Bottleneck
Small file on disk	Disk latency (seek time)
Large file in same rack	Network bandwidth (e.g., rack switch)
Remote disk across racks	Latency still often dominated by disk seek

7 The Four Big Design Ideas

1) Scale Out, Not Up
Prefer many commodity machines over one massive server. This improves cost-efficiency and fault tolerance.

2) Seamless Scalability
Performance should improve roughly linearly with additional machines for suitable workloads.

3) Move Compute to Data
Send code to where data lives instead of moving huge datasets over the network.

4) Favor Sequential Access
Design storage and execution around scans, not random reads, to avoid large seek overheads.

Scale-out intuition: 100 disks at 200 MB/s each provide ~20 GB/s aggregated disk bandwidth when work is parallelized well.

8 Abstractions for Distributed Execution

At very large scale, programmers cannot manually manage failures, synchronization, scheduling, and data movement.

Machine Failures
In large clusters, failures are normal. Systems must retry tasks and recover automatically.

Synchronization
Workers run at different speeds. Coordination primitives (e.g., barriers) are needed between phases.

Programming Complexity
Low-level distributed programming is hard. Abstractions reduce bugs and development time.

Right abstraction: The developer specifies *what* to compute (e.g., map/reduce style logic), while the framework handles *how* to execute it robustly at scale.

```
Programmer code   → map/reduce style operations
Framework handles → partitioning, scheduling, retries, data movement, aggregation
```

9 Comparison Summary

Design Choice	Preferred in Big Data Systems	Reason
Scaling strategy	Scale out (many machines)	Better aggregate bandwidth and fault tolerance per dollar
Data movement	Move computation to data	Network is expensive bottleneck at scale
Disk access pattern	Sequential	Avoid repeated seek latency penalties
Programming model	High-level abstraction + runtime	Handles failures/synchronization automatically

10 Exam-Style Q&A

Q: Server A reads a file from Server B's disk in the same rack. What path does data follow?

> Data is first loaded from B's disk into B's memory, then transferred through the rack switch to A. No datacenter switch is needed for same-rack transfer.

Q: For a very small file read from disk, what dominates total time?

> Disk latency dominates, because seek/startup cost is large relative to tiny data-transfer volume.

Q: For a very large file transfer across machines in one rack, what is often the bottleneck?

> Network bandwidth (e.g., rack switch bandwidth), since sustained transfer rate matters more than fixed startup delay.

Q: Why do frameworks like Hadoop/Spark use high-level abstractions?

> They let programmers express logic while the framework manages partitioning, retries, synchronization, and result aggregation across many machines.

11 Key Takeaways

1. **The datacenter is the computer:** reason about computation at cluster scale, not just one machine.
2. **Hardware constraints drive software design:** latency, bandwidth, and storage hierarchy directly shape algorithms and execution plans.
3. **Scale-out and data locality are central:** move compute to data and aggregate many commodity nodes for throughput.
4. **Abstractions are essential:** distributed runtimes hide operational complexity and make large-scale processing practical.